

VOLUME II

Build Real Full-Stack Apps With AI

From AI-Assisted Website Builder to AI Application Developer.

PART 1: From Website to Web Application

01. What Changes After Volume I?

Volume I built websites. Now, we build software. The core difference is the **User** and their **Data**.

AIWEBDEV PROMPT 2.01

THE TRANSITION DIAGNOSTIC

USE WHEN

Understanding what an existing website needs.

CONTEXT TO ATTACH

@project @frontend-files

RUN INSIDE

Antigravity Agent

RECOMMENDED ROLE

Strong reasoning model

DO NOT ATTACH

None

NEXT

Prompt 2.02 Requirements

Inspect my existing project.

Do not modify anything.

Explain which parts are currently:

1. static frontend
2. interactive frontend
3. server-side
4. API-related
5. database-related
6. authentication-related

Then tell me what would need to be added to turn this website into a real web application. Explain it as if I have completed AIWebDev Volume I.

PART 2 & 3: Requirements & Architecture

02 & 03. Antigravity as Technical Architect

Never type "Build me a SaaS." Teach your AI to **Plan** → **Build**. Use this prompt to design your system architecture before writing a single line of code.

AIWEBDEV PROMPT 2.02

SYSTEM ARCHITECTURE PLAN

USE WHEN

Starting a new full-stack project feature.

CONTEXT TO ATTACH

@project @requirements

RUN INSIDE

Antigravity Agent

RECOMMENDED ROLE

Strong reasoning model

DO NOT ATTACH

Database credentials

NEXT

Review & Approve Plan

Act as the technical architect for this project.

DO NOT write implementation code yet.

PROJECT: [Insert project idea]

USERS: [List user types]

CORE FEATURES: [List main features]

Before implementation:

1. Inspect the existing repository.
2. Identify the current framework and dependencies.
3. Propose the simplest production-appropriate architecture.
4. Identify frontend responsibilities vs server responsibilities.
5. Design the database entities and relationships.
6. Define authentication requirements & authorization boundaries.
7. Break implementation into phases and identify major security risks.

Prefer the simplest architecture. Explain the plan in beginner-friendly English.

Wait for my approval before implementation.

PART 3.5: App Router Mapping

04b. Next.js Routing Workflow

Before building pages, map out the Next.js App Router structure. Understand the difference between public pages, authenticated dashboards, and API routes.

AIWEBDEV PROMPT 2.02b

APP ROUTER MAPPER

COPY THIS INTO ANTIGRAVITY:

Based on our approved architecture, please map out the entire Next.js App Router structure (`app/` directory).

For each route, specify:

1. The exact folder path (e.g. `app/(dashboard)/layout.tsx`)
2. Is it a Client Component (`use client`) or Server Component?
3. Is it public or protected by authentication?
4. What data does it fetch?

Do not create the files yet. Provide the map for my approval.

PART 4 & 5: Databases Without Fear

06. Let AI Design the Schema — But Review It

Databases are hard to change later. Never let AI dump SQL into your project without explaining the relationships and security models first.

AIWEBDEV PROMPT 2.03

DATABASE SCHEMA DESIGNER

USE WHEN

Generating SQL for a new feature.

CONTEXT TO ATTACH

@approved-architecture @supabase-config

RUN INSIDE

Antigravity Agent

RECOMMENDED ROLE

Strong reasoning model

DO NOT ATTACH

None

NEXT

Run SQL in Supabase Dashboard

We need a Supabase database for this application.

Do not create anything yet.

Based on the approved requirements, propose the database schema.

For every table show:

- purpose
- columns & data types
- primary key & foreign keys
- required fields & default values
- relationships & ownership model
- required indexes
- RLS requirements

Also identify data that should NOT be stored.

Explain the schema before generating SQL.

Wait for my approval.

PART 6: Authentication

07. Building Authentication (Step 1 & 2)

Authentication handles "Who are you?" Don't use a magic prompt. Use the engineering loop: Inspect, Plan, Implement, Test.

AIWEBDEV PROMPT 2.04

AUTH FLOW — INSPECT

USE WHEN	RECOMMENDED ROLE
Adding auth to an existing codebase.	Fast model
CONTEXT TO ATTACH	DO NOT ATTACH
@project @routes	None
RUN INSIDE	NEXT
Antigravity Agent	Prompt 2.05 Plan

Inspect the existing authentication architecture of this project.

Do not modify anything.

Determine:

- whether authentication already exists
- which provider is being used
- how sessions are managed
- where login/signup UI exists
- how protected routes currently work

Explain the current authentication flow. Identify anything missing or insecure.
Do not implement yet.

AIWEBDEV PROMPT 2.05

AUTH FLOW — PLAN

USE WHEN	RECOMMENDED ROLE
Designing the auth architecture.	Strong reasoning model
CONTEXT TO ATTACH	DO NOT ATTACH
@inspection-results	None
RUN INSIDE	NEXT
Antigravity Agent	Prompt 2.06 Implement

Design the authentication implementation for this application.

Requirements:

- Signup, Login, Logout using Supabase
- Persistent session & Protected dashboard
- Useful error messages and loading states

Show:

FILES TO CREATE

FILES TO MODIFY

ENVIRONMENT VARIABLES

SECURITY CONSIDERATIONS

TEST PLAN

Wait for approval.

Authentication Strategies

Magic Links: Highly secure, no passwords to lose, but users must open their email. Best for B2B.

OAuth (Google/GitHub): 1-click friction, but requires developer console setup. Best for B2C.

Email/Password: Traditional, high friction, requires password reset flows.

07. Building Authentication (Step 3 & 4)

Once the plan is approved, execute it. If it fails, use the debug prompt.

AIWEBDEV PROMPT 2.06

AUTH FLOW — IMPLEMENT & TEST

USE WHEN

Writing the actual auth code.

CONTEXT TO ATTACH

@approved-auth-plan @ui-components

RUN INSIDE

Antigravity Agent

RECOMMENDED ROLE

Strong builder model

DO NOT ATTACH

.env.local with real secrets

NEXT

Manual Browser Test

The authentication implementation plan is approved. Implement it now.

Requirements:

- Preserve the existing UI. Reuse existing components where possible.
- Never hard-code credentials. Keep private credentials server-side.
- Add appropriate error handling (e.g. invalid password, email exists).
- Do not claim the implementation works until it has been tested.

After implementation:

1. List every file you changed and why.
2. Tell me exactly which environment variables I must configure locally.
3. Tell me exactly how to test the feature in my browser.

AIWEBDEV PROMPT 2.07

AUTH FLOW — DEBUG

USE WHEN

Fixing a broken login/signup.

CONTEXT TO ATTACH

@auth-files @browser-console-errors

RUN INSIDE

Antigravity Agent

RECOMMENDED ROLE

Strong reasoning model

DO NOT ATTACH

None

NEXT

Re-test in browser

Authentication is not working correctly.

Do NOT rewrite the authentication system.
Trace the existing flow.

Inspect:

1. browser console & network requests
2. authentication response
3. session creation & cookies
4. middleware/route protection

Use the actual error evidence available in this project.
Identify the root cause before modifying code. Apply the smallest appropriate fix.
After fixing it, explain how I can verify the resolution.

PART 9: Row Level Security

12. Build RLS With Antigravity

Hiding Rahul's appointment in the frontend is not security. You must lock the database itself. RLS is how you do it.

AIWEBDEV PROMPT 2.08

RLS ARCHITECTURE AUDIT

USE WHEN Securing tables before launch.	RECOMMENDED ROLE Strong independent reviewer
CONTEXT TO ATTACH @database-schema @supabase/policies	DO NOT ATTACH None
RUN INSIDE Antigravity Agent	NEXT Prompt 2.09 Apply Policies

Perform a Supabase authorization review.

Goal: A normal authenticated user must only be able to access records they are authorized to access.

Do not modify the database yet.

For every relevant table:

1. identify who owns each row
2. identify who may SELECT, INSERT, UPDATE, and DELETE it
3. identify admin/business-owner exceptions
4. identify required RLS policies

Look specifically for: missing RLS, overly broad policies, ownership bypasses, and service-role misuse.

Explain the proposed policies first. Generate the migration only after approval.

PART 11: File Uploads

14. Supabase Storage

Handling files correctly means knowing the difference between a Public Bucket (logos) and a Private Bucket (invoices).

AIWEBDEV PROMPT 2.09**STORAGE INTEGRATION PLAN**

USE WHEN

Adding profile pictures or document uploads.

CONTEXT TO ATTACH

@project @schema @file-upload-ui

RUN INSIDE

Antigravity Agent

RECOMMENDED ROLE

Strong builder model

DO NOT ATTACH

None

NEXT

Approve Storage Plan

We need to implement file uploads using Supabase Storage.

Do not write code yet. First, define the storage architecture:

1. Do we need a public or private bucket?
2. What are the file size and type restrictions?
3. How will the application handle the upload securely?
4. How is the file URL linked to the database record?
5. What Storage RLS policies are required?

Provide the implementation plan, the required bucket configuration, and the required SQL for policies. Wait for approval.

PART 13 & 14: APIs & Documentation

17. Stop Letting AI Guess APIs

APIs change constantly. If you tell an AI to "integrate Mailchimp," it might hallucinate code from 2021. Give it the docs.

AIWEBDEV PROMPT 2.10

DOCUMENTATION-DRIVEN INTEGRATION

USE WHEN

Connecting external APIs safely.

CONTEXT TO ATTACH

@project @api-docs (or paste URL)

RUN INSIDE

Antigravity Agent

RECOMMENDED ROLE

Strong reasoning model

DO NOT ATTACH

API Keys

NEXT

Approve Integration Plan

I want to integrate this service.

Do not rely solely on memory. Use the documentation/context I provide as the source of truth.

First determine:

1. current authentication method (e.g. Bearer token)
2. required package/API endpoints
3. required environment variables (server vs client)
4. request and response format
5. errors we must handle

Compare that against my existing project. Then propose the implementation plan.

Do not modify code until the integration plan is clear.

PART 19 & 20: Payments (Razorpay)

22. How Online Payments Actually Work

Never trust a "payment-success" page as proof that somebody paid. Hackers can just type that URL into their browser. Payments require server-side verification.

AIWEBDEV PROMPT 2.11

PAYMENT FLOW ARCHITECTURE

USE WHEN

Integrating Razorpay or Stripe.

CONTEXT TO ATTACH

@project @payment-docs @schema

RUN INSIDE

Antigravity Agent

RECOMMENDED ROLE

Strong reasoning model

DO NOT ATTACH

Live Secret Keys

NEXT

Approve Payment Architecture

I need to integrate a payment provider (e.g. Razorpay).

Do not implement yet. Design the secure payment architecture.

Ensure the design covers:

1. Server-created orders/intents (never client-side pricing).
2. The checkout UI flow.
3. Server-side verification (Webhooks or Server Actions).
4. Database updates (granting access ONLY after server verification).
5. Environment variables (Test vs Live, Public vs Secret).

Identify where a malicious user might try to bypass payment, and explain how our architecture prevents it. Wait for approval.

PART 21: Webhooks

24. What Is a Webhook?

Instead of your app asking "Did they pay yet?", the provider pings your server and says "Payment complete!" This ping is a Webhook.

AIWEBDEV PROMPT 2.12

WEBHOOK IMPLEMENTATION

USE WHEN

Building server-side listeners for events.

CONTEXT TO ATTACH

@project @payment-architecture

RUN INSIDE

Antigravity Agent

RECOMMENDED ROLE

Strong builder model

DO NOT ATTACH

None

NEXT

Test with Provider CLI

Implement the webhook endpoint based on our approved payment architecture.

Requirements:

1. The endpoint MUST verify the webhook signature/authenticity using the provider's secret key. Do not accept unsigned requests.
2. Ensure the endpoint handles idempotency (don't grant access twice if the provider sends the webhook twice).
3. Update the database securely based on the event payload.
4. Add clear server logging for debugging.

Provide instructions on how I can test this webhook locally using the provider's CLI or testing tools.

PART 22: Build a Real Access System

25. Digital Product Vault Mission

This is the capstone pattern: commerce, auth, and authorization combined into a single flow.

AIWEBDEV PROMPT 2.13

DIGITAL VAULT ARCHITECTURE

USE WHEN

Planning a gated content system.

CONTEXT TO ATTACH

@project @payment-webhook @auth-logic

RUN INSIDE

Antigravity Agent

RECOMMENDED ROLE

Strong reasoning model

DO NOT ATTACH

None

NEXT

Approve Vault Plan

I want to build a Digital Product Vault where users can only access specific resources after a verified payment.

Do not write code yet. Design the architecture.

Requirements:

1. Product Page → Checkout → Verified Payment → Account Creation → Entitlement → Protected Resources.
2. Define how the database will track which user bought which product (Entitlements table).
3. Define the RLS policies required to ensure users cannot download resources they did not buy.
4. Explain how the webhook from our payment provider updates this Entitlements table securely.

Provide the schema changes and architectural flow for approval.

PART 23 & 25: Connect Everything

28. Vercel + Render + Supabase

A true full-stack production architecture looks like this:

```
USER → www.example.com
      ↓
VERCEL (Next.js App)
      ↓ HTTPS API call
api.example.com → RENDER (FastAPI API)
      ↓
SUPABASE (Database) & EXTERNAL APIS
```

PART 26: Production Environment Matrix

Not every variable belongs everywhere. Map it out:

Variable	Local	Vercel	Render
Supabase URL & Client Key	✓	✓	Maybe
Service-role / Payment Key	If server	Server Only	Server Only
Backend API_URL	✓	✓	—

PART 28: Full-Stack Debugging

31. Trace the Entire Request

When a button fails in production, do not say "fix it". Trace the error to give AI the right context.

THE 4-LAYER DEBUGGING METHOD

- 1. Frontend (Browser Console):** Did the React component crash before even sending data? Check F12 Console.
- 2. Network (Browser Network Tab):** Did the request fail? (e.g., CORS 403, Not Found 404). Check Payload & Response.
- 3. Server (Vercel/Render Logs):** Did the backend crash while processing? Check terminal or platform logs (Error 500).
- 4. Database (Supabase Logs/RLS):** Did the query get blocked by Row Level Security?

AIWEBDEV PROMPT 2.14

FULL-STACK TRACE

USE WHEN

Diagnosing a complex production failure.

CONTEXT TO ATTACH

@frontend-route @backend-route @logs

RUN INSIDE

Antigravity Agent

RECOMMENDED ROLE

Strong reasoning model

DO NOT ATTACH

None

NEXT

Implement recommended fix

My application is deployed to production, but I am facing an issue.

Symptom: User clicks 'Book Appointment' and nothing happens.

Here are the deployment logs / Browser Network tab errors:

[PASTE LOGS HERE]

Do not rewrite the code blindly. Trace the flow:

1. Did the browser send the request correctly?
2. Did Vercel route it to the backend?
3. Did the backend receive the auth tokens?
4. Did Supabase reject the insert due to RLS?

Identify exactly where the chain breaks, and apply the smallest appropriate fix.

PART 30 & 31: Multi-Agent Workflows

33. Stop Asking One Agent to Do Everything

A single AI model cannot plan architecture, write React, configure Postgres, and audit security all in one prompt. You must route the tasks.

1. **The Architect (Reasoning Model):** "Design this feature and wait for approval."
2. **The Builder (Fast/Coding Model):** "Implement the approved plan exactly."
3. **The Reviewer (Independent Model):** "Audit the builder's code before I deploy."
4. **The Debugger (Reasoning Model):** "Trace this stack trace and find the root cause."

PART 32: Agent Rules

35. Persistent Project Instructions

Give your AI environment rules like: *"Never expose secrets to the frontend. Reuse existing components. Never fabricate test success."*



PART 34 & 35: AI Security Review

38. Exact Pre-Launch Security Prompt

Never launch a full-stack app without a dedicated security sweep. Treat AI as a malicious auditor.

AIWEBDEV PROMPT 2.15

PRE-LAUNCH SECURITY AUDIT

USE WHEN Final check before sending traffic to your app.	RECOMMENDED ROLE Strong independent reviewer
CONTEXT TO ATTACH @project @supabase-schema	DO NOT ATTACH None
RUN INSIDE Antigravity Agent	NEXT Address CRITICAL findings

Act as a strict cybersecurity auditor. Perform a pre-launch security review of this codebase.

Do not modify anything yet.

Look specifically for:

- Secrets exposed in frontend code/variables.
- Missing authorization checks on server routes.
- RLS disabled or misconfigured in Supabase.
- Trusting user IDs from the client instead of the session.
- Unvalidated inputs or unrestricted uploads.

Output your findings categorized by: CRITICAL, HIGH, MEDIUM, LOW.

For each finding, provide: The finding → the exact file/evidence → the impact → the recommended fix. Wait for my instructions.

VOLUME II CAPSTONE PROJECTS

You now have the exact prompts and workflows to build real software. It is time to prove it. Choose one of the following architectures to build from scratch.

Project 1: Clinic System

- Auth & RLS
- Book & Cancel Appointments
- Admin View

Project 2: Lead CRM

- Protected Dashboards
- Status Pipelines
- Search & Filter

Project 3: AI SaaS

- External LLM APIs
- Usage Limits
- Graceful AI Errors

Project 4: Digital Vault

- Razorpay Webhooks
- Server-side verification
- Entitlement logic



47. Volume II Graduation

You are now an AI Application Developer.

- ✓ I can ask AI to design an architecture.
- ✓ I can build authentication.
- ✓ I understand Row Level Security.
- ✓ I can integrate payment webhooks securely.
- ✓ I can trace a bug from browser to database.
- ✓ I can use multiple agents intelligently.
- ✓ I can verify instead of assuming AI is correct.

NEXT → VOLUME III

Turn Your Skills Into a Business

Finding niches, factual audits, proposals, delivery, and retainers.

